

Universidad Mariano Gálvez de Guatemala
Facultad de Ingeniería en Sistemas de Información
Bases de Datos I

AutoAudit

Sistema de auditoría automatizada para PostgreSQL 18
Documentación final del proyecto

Integrantes	José Mario Rosales Palma	0905-24-17488
	Jefersón Oslee Cermeño Pineda	0905-24-12088
	Andrea Fernanda Ronquillo Godoy	24-15384
	Ennio Cristobal Felipe Trigueros	1690-19-4503
Catedrático	Ing. Edgar René Ornelis Hoil	
Fecha	Noviembre de 2026	

Contenido

1. Descripción general y arquitectura
2. Diagrama entidad-relación
3. Diccionario de datos
4. Manual de configuración y activación por tabla
5. Instalación y despliegue de triggers
6. Módulo de consultas y reportes
7. Seguridad y control de acceso
8. Pruebas de validación y rendimiento
9. Limitaciones conocidas
10. Estructura del repositorio

1. Descripción general y arquitectura

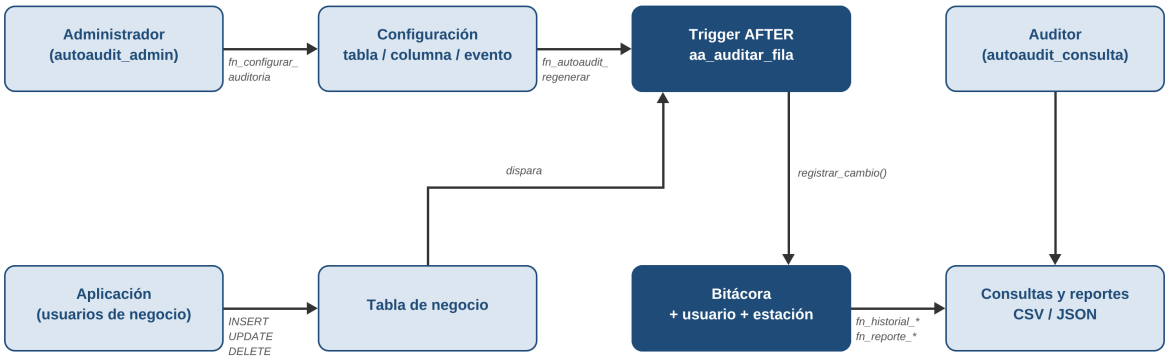
AutoAudit es un sistema de auditoría para PostgreSQL 18 que registra automáticamente los cambios que se hacen sobre las tablas de negocio de una base de datos. El administrador elige qué tablas, qué columnas y qué operaciones (INSERT, UPDATE, DELETE) quiere vigilar; a partir de esa configuración el sistema genera los triggers necesarios y, desde ese momento, cada cambio queda guardado en una bitácora con el valor anterior, el valor nuevo, el usuario que lo hizo, la estación de trabajo y la fecha y hora.

La idea central es que nadie tenga que escribir triggers a mano. La configuración vive en tablas propias del sistema, y un generador lee esa configuración y crea o reemplaza el trigger de cada tabla. La aplicación de negocio sigue trabajando igual que antes: no necesita permisos sobre la bitácora ni sabe que está siendo auditada.

1.1 Componentes

Módulo	Scripts	Qué hace
Estructura base	01, 03	Crea las siete tablas del modelo y agrega las restricciones UNIQUE y CHECK que necesita la configuración. Inserta los tres eventos INSERT, UPDATE y DELETE.
Diccionario y configuración	04, 05	Permite ver las tablas y columnas disponibles, seleccionar qué se audita, activar o desactivar eventos y validar que la configuración siga siendo coherente con el esquema real.
Triggers y bitácora	07, 08	Función que registra cada cambio, guardias que protegen el historial y generador que crea los triggers a partir de la configuración.
Consultas y reportes	10, 11, 12	Consultas históricas parametrizadas, reportes de resumen, exportación a CSV y JSON, índices y rol de solo lectura.
Pruebas	06, 09, 13	Casos de prueba de configuración, de triggers y de validación completa con mediciones de rendimiento.
Ejemplo	14	Recorrido completo con una tabla de clientes, desde la configuración hasta los reportes.

1.2 Flujo de funcionamiento

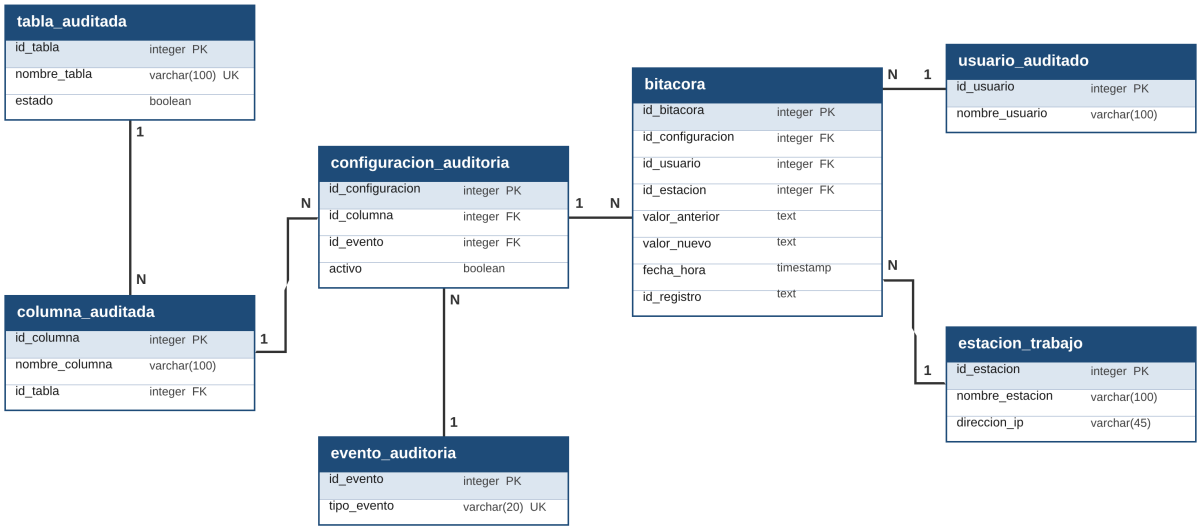


El administrador consulta el diccionario de datos para ver qué tablas y columnas existen, registra la configuración con `fn_configurar_auditoria` y en la misma transacción llama a `fn_autoaudit_regenerar`. El generador instala en la tabla un trigger `AFTER ... FOR EACH ROW` llamado `aa_auditar_fila` que apunta a la función común `autoaudit_interno.registrar_cambio()`. Cuando la aplicación inserta, modifica o elimina filas, esa función revisa qué columnas están activas para el evento y escribe una fila en la bitácora por cada columna afectada. Los auditores consultan el historial con las funciones del módulo de consultas o exportan los reportes.

Cada fila de la bitácora apunta a una configuración (que a su vez identifica tabla, columna y evento), a un usuario y a una estación. Así no se repiten nombres en cada registro y la cadena de relaciones permite reconstruir todo el contexto del cambio.

2. Diagrama entidad-relación

El modelo tiene siete tablas en el esquema `public`. Las cuatro primeras guardan la configuración (qué se audita) y las otras tres guardan el resultado (qué pasó, quién y desde dónde).



Diagramas en Canva: https://www.canva.com/design/DAHZt1u_StU/view

2.1 Relaciones

Relación	Cardinalidad	Significado
tabla_auditada → columna_auditada	1 : N	Una tabla registrada tiene varias columnas seleccionables.
columna_auditada → configuracion_auditoria	1 : N	Cada columna puede tener hasta tres configuraciones, una por evento.
evento_auditoria → configuracion_auditoria	1 : N	Cada evento (INSERT, UPDATE, DELETE) se usa en muchas configuraciones.
configuracion_auditoria → bitacora	1 : N	Cada registro de bitácora corresponde a una columna y un evento concretos.
usuario_auditado → bitacora	1 : N	Un usuario de base de datos genera muchos registros.
estacion_trabajo → bitacora	1 : N	Una estación (IP de origen) aparece en muchos registros.

La combinación (id_columna, id_evento) es única en `configuracion_auditoria`, por lo que configurar dos veces lo mismo no duplica filas. Desactivar una configuración nunca la borra: se cambia `activo` a falso y la bitácora conserva su referencia.

3. Diccionario de datos

3.1 tabla_auditada

Tablas de negocio que se registraron para auditoría.

Campo	Tipo	Restricciones	Descripción
id_tabla	integer	PK, IDENTITY	Identificador de la tabla registrada.
nombre_tabla	varchar(100)	NOT NULL, UNIQUE	Nombre de la tabla en el esquema public.
estado	boolean	NOT NULL, DEFAULT true	Si es falso, la tabla completa deja de auditarse aunque sus columnas sigan configuradas.

3.2 columna_auditada

Columnas de cada tabla registrada que pueden auditarse.

Campo	Tipo	Restricciones	Descripción
id_columna	integer	PK, IDENTITY	Identificador de la columna.
nombre_columna	varchar(100)	NOT NULL	Nombre de la columna en la tabla de negocio.
id_tabla	integer	NOT NULL, FK → tabla_auditada	Tabla a la que pertenece.
UNIQUE (id_tabla, nombre_columna)			

3.3 evento_auditoria

Catálogo fijo con las tres operaciones auditables.

Campo	Tipo	Restricciones	Descripción
id_evento	integer	PK, IDENTITY	Identificador del evento.
tipo_evento	varchar(20)	NOT NULL, UNIQUE, CHECK (INSERT, UPDATE, DELETE)	Operación que se audita.

3.4 configuracion_auditoria

Une columna y evento: cada fila dice "audita esta columna cuando ocurra este evento".

Campo	Tipo	Restricciones	Descripción
id_configuracion	integer	PK, IDENTITY	Identificador de la configuración.
id_columna	integer	NOT NULL, FK → columna_auditada	Columna configurada.

id_evento	integer	NOT NULL, FK → evento_auditoria	Evento configurado.
activo	boolean	NOT NULL, DEFAULT true	Permite apagar una combinación sin borrarla.
UNIQUE (id_columna, id_evento)			

3.5 usuario_auditado

Usuarios de base de datos que han hecho cambios. Se llena automáticamente desde el trigger.

Campo	Tipo	Restricciones	Descripción
id_usuario	integer	PK, IDENTITY	Identificador del usuario.
nombre_usuario	varchar(100)	NOT NULL	Login de la sesión (`session_user`), aunque se haya usado SET ROLE.

3.6 estacion_trabajo

Origen de la conexión. También se llena automáticamente.

Campo	Tipo	Restricciones	Descripción
id_estacion	integer	PK, IDENTITY	Identificador de la estación.
nombre_estacion	varchar(100)	NOT NULL	`IP:<dirección>` o `SIN_IP_DISPONIBLE` cuando la conexión es por socket local.
direccion_ip	varchar(45)	NULL permitido	Resultado de `inet_client_addr()`; admite IPv6.

3.7 bitacora

Historial de cambios. Una fila por cada columna configurada que cambió en cada operación.

Campo	Tipo	Restricciones	Descripción
id_bitacora	integer	PK, IDENTITY	Correlativo del registro.
id_configuracion	integer	NOT NULL, FK → configuracion_auditoria	Identifica tabla, columna y evento.
id_usuario	integer	NOT NULL, FK → usuario_auditado	Quién hizo el cambio.
id_estacion	integer	NOT NULL, FK → estacion_trabajo	Desde dónde se hizo.
valor_anterior	text	NULL permitido	Valor antes del cambio. NULL en INSERT.

valor_nuevo	text	NULL permitido	Valor después del cambio. NULL en DELETE.
fecha_hora	timestamp	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Momento exacto (`clock_timestamp()`, hora de Guatemala).
id_registro	text	NULL permitido	Llave primaria de la fila afectada en formato JSON, por ejemplo `{"id_cliente": 3}`. Si cambió la PK: `{"anterior": {...}, "nuevo": {...}}`. NULL si la tabla no tiene PK.

Índices agregados por el módulo de consultas sobre `bitacora`:

Índice	Columnas	Uso
ix_bitacora_usuario_fecha	(id_usuario, fecha_hora, id_bitacora)	Historial por usuario
ix_bitacora_configuracion_fecha	(id_configuracion, fecha_hora, id_bitacora)	Historial por columna y por tabla
ix_bitacora_fecha	(fecha_hora, id_bitacora)	Línea de tiempo y rangos de fechas

3.8 Vistas y funciones

Objeto	Módulo	Descripción
v_dic_tablas	04	Tablas de negocio disponibles, número de columnas y si ya están auditadas.
fn_dic_columnas(tabla)	04	Columnas de una tabla con tipo, nulos, PK y eventos configurados.
v_configuracion_actual	05	Configuración completa con nombres de tabla, columna y evento.
fn_configurar_auditoria(tabla, columnas[], eventos[])	05	Registra tabla, columnas y eventos. NULL significa todas las columnas o todos los eventos.
fn_cambiar_estado_evento(tabla, columna, evento, activo)	05	Activa o desactiva una combinación puntual.
fn_deshabilitar_tabla(tabla)	05	Deshabilita la tabla completa.
fn_validar_configuracion()	05	Detecta configuraciones que apuntan a tablas o columnas que ya no existen.
fn_autoaudit_regenerar(tabla)	08	Crea, actualiza o retira el trigger según la configuración. Sin argumento procesa todas.
fn_autoaudit_retirar(tabla)	08	Quita el trigger sin tocar configuración ni historial.
autoaudit_interno.registrar_cambio()	07	Función del trigger que escribe en la bitácora (SECURITY DEFINER).

v_bitacora_detalle	10	Bitácora con todos los nombres resueltos. Base de todas las consultas.
fn_historial_usuario, fn_historial_columna, fn_historial_registro, fn_linea_tiempo	10	Consultas históricas parametrizadas (sección 6).
fn_reporte_por_usuario, fn_reporte_por_columna, fn_reporte_actividad_diaria, fn_reporte_estado_auditoria	11	Reportes de resumen (sección 6).

4. Manual de configuración y activación por tabla

Todo lo de esta sección lo hace un usuario con el rol `autoaudit_admin`. Las tablas de configuración no aceptan INSERT ni UPDATE directos; siempre se trabaja con las funciones. Como ejemplo se usa una tabla `cliente` con las columnas id_cliente, nombre, limite_credito, telefono y categoria.

4.1 Revisar el diccionario

```
SELECT * FROM public.v_dic_tablas;
SELECT orden, nombre_columna, tipo_dato, es_pk
  FROM public.fn_dic_columnas('cliente');
```

La primera consulta muestra las tablas de negocio disponibles (las del propio sistema no aparecen). La segunda muestra las columnas, su tipo y cuál es la llave primaria. Es buena idea confirmar que la tabla tiene PK, porque es lo que permite identificar después qué fila cambió.

4.2 Activar la auditoría de una tabla

Se configura y se genera el trigger dentro de la misma transacción. Si cualquiera de los dos pasos falla, el ROLLBACK deja todo como estaba.

```
BEGIN;
-- limite_credito y categoria en los tres eventos
SELECT public.fn_configurar_auditoria('cliente', ARRAY['limite_credito','categoria']);
-- nombre solo al insertar y al borrar
SELECT public.fn_configurar_auditoria('cliente', ARRAY['nombre'],
ARRAY['INSERT','DELETE']);
-- instalar el trigger
SELECT * FROM public.fn_autoaudit_regenerar('cliente');
COMMIT;
```

Para auditar todas las columnas en todos los eventos basta `SELECT fn_configurar_auditoria('cliente');`. La función valida que la tabla y las columnas existan (errores 42P01 y 42703) y aplica todo o nada.

4.3 Verificar el estado

```
SELECT * FROM public.v_configuracion_actual WHERE nombre_tabla = 'cliente';
SELECT * FROM public.fn_reporte_estado_auditoria();
```

El reporte de estado indica, por tabla, si está habilitada, si sigue existiendo, cuántas columnas están activas, qué eventos se auditan, si el trigger está instalado y cuántos registros tiene en la bitácora. En el ejemplo de la sección 6 el resultado es: *cliente | t | t | 3 | DELETE, INSERT, UPDATE | t | 15*.

4.4 Cambiar o desactivar

Necesidad	Instrucción
Dejar de auditar un evento de una columna	`fn_cambiar_estado_evento('cliente','limite_credito','UPDATE', false)`
Volver a activarlo	`fn_cambiar_estado_evento('cliente','limite_credito','UPDATE', true)`
Agregar columnas nuevas	`fn_configurar_auditoria('cliente', ARRAY['telefono'])`
Deshabilitar toda la tabla	`fn_deshabilitar_tabla('cliente')`
Quitar el trigger sin tocar la configuración	`fn_autoaudit_retirar('cliente')`

Desactivar algo tiene efecto inmediato, porque la función de registro revisa la configuración en cada operación. En cambio, agregar un evento que el trigger todavía no escucha, registrar una tabla nueva, cambiar la PK o rehabilitar una tabla requiere volver a llamar a `fn_autoaudit_regenerar`. Por costumbre conviene llamarlo siempre después de cualquier cambio de configuración, en la misma transacción:

```
BEGIN;
SELECT public.fn_deshabilitar_tabla('cliente');
SELECT * FROM public.fn_autoaudit_regenerar('cliente');
COMMIT;
```

Ninguna de estas operaciones borra historial. Si una tabla se deshabilita, sus registros anteriores siguen consultables (prueba D16).

4.5 Después de cambios de estructura

Si se altera una tabla auditada (se borra o renombra una columna, se cambia la PK), primero hay que revisar `SELECT * FROM fn_validar_configuracion();`, desactivar lo que ya no exista y regenerar antes de volver a operar sobre la tabla.

5. Instalación y despliegue de triggers

5.1 Requisitos

Ubuntu 26.04 LTS con servidor y cliente PostgreSQL 18. Instala como el superusuario postgres y utiliza cuentas normales para la aplicación. Si los paquetes no están en tus repositorios, consulta el repositorio oficial de PostgreSQL para Ubuntu.

```
sudo apt install postgresql-18 postgresql-client-18
sudo systemctl start postgresql
sudo -u postgres psql -X -d postgres -c 'SHOW server_version;'
```

5.2 Instalación automática

Desde la raíz del proyecto, utiliza el socket Unix local o conecta por TCP con contraseña.

```
bash instalar.sh autoaudit
bash instalar.sh --password 127.0.0.1 autoaudit
```

El modo TCP requiere una terminal interactiva y autenticación SCRAM en las bases postgres y de destino. psql pide la contraseña sin eco. El instalador ignora PGPASSWORD y el archivo de contraseñas. Para un host remoto se exige TLS con certificado y nombre verificados. Define PGSSLROOTCERT si necesitas indicar la CA y PGPORT para cambiar el puerto.

5.3 Validación de la estructura

El instalador comprueba las versiones del cliente y del servidor, el propietario de la base y los permisos CREATE de los esquemas. En una base existente verifica las siete tablas del modelo, sus columnas, identidades y claves. Una estructura base parcial o incompatible detiene el despliegue.

Los módulos se aplican en orden 01, 03, 04, 05, 07, 08, 10 y 11 dentro de una transacción en la base de destino. El paso 01 se ejecuta si falta el modelo base completo. La creación de la base es previa a la transacción. Una reinstalación compatible conserva los datos y la configuración.

5.4 Roles

Rol	Para quién	Cómo se asigna
autoaudit_admin	Configura, genera triggers y consulta.	GRANT autoaudit_admin TO usuario;
autoaudit_consulta	Consulta historial y reportes.	GRANT autoaudit_consulta TO usuario;
Roles de negocio	Operan las tablas de negocio.	Permisos de negocio según la aplicación.

5.5 Despliegue de los triggers

El administrador selecciona tablas, columnas y eventos desde las vistas y funciones de configuración. Aplica la configuración con el generador dentro de la misma transacción.

```
SELECT * FROM public.fn_autoaudit_regenerar('cliente');
SELECT * FROM public.fn_autoaudit_regenerar();
```

El generador lee la configuración activa, obtiene la clave primaria y crea un trigger aa_auditar_fila por tabla con la unión de los eventos activos. Si ya existe, lo reemplaza. Si la tabla queda sin eventos activos, lo retira. Ante una colisión con un trigger ajeno, aborta la operación.

El reporte anterior indica si el objeto existe. La consulta siguiente distingue presencia y habilitación en sesiones normales y expone el modo del trigger.

```
SELECT * FROM public.fn_reporte_estado_triggers();
```

Los cambios durante una pausa de auditoría quedan fuera de la bitácora. Reactivar el trigger permite registrar operaciones posteriores. Mantén activos los guardias de escritura de la bitácora.

5.6 Verificar la instalación

El instalador excluye pruebas, exportación y demostración. Ejecuta las pruebas en bases desechables y conserva las salidas en un directorio nuevo. Los scripts 09 y 13 terminan con ROLLBACK. Ejecuta 06 en otra base porque recrea empleado y deja una configuración obsoleta.

```
bash instalar.sh autoaudit_pruebas
sudo -u postgres psql -X -d autoaudit_pruebas \
  -f sql/09_pruebas_triggers.sql
sudo -u postgres psql -X -d autoaudit_pruebas \
  -f sql/13_pruebas_validacion.sql
```

Consulta docs/REVISION_FINAL.md para los casos de instalación por TCP, reinstalación, estructuras parciales, estado de triggers y exportación. Las salidas incluidas son históricas. La validación de esta revisión en Ubuntu 26.04 LTS con PostgreSQL 18.6 está pendiente.

6. Módulo de consultas y reportes

Todas las consultas reciben parámetros con nombre y valores por defecto, así que se pueden usar solo los filtros necesarios. Las fechas siguen la convención *desde inclusivo, hasta exclusivo*: para todo noviembre se usa `'2026-11-01'` y `'2026-12-01'`. Si *hasta* no es mayor que *desde*, la función devuelve error. Los resultados siempre salen en orden cronológico.

Las consultas no filtran por `activo`: un cambio registrado sigue siendo válido aunque después se haya desactivado esa columna o deshabilitado la tabla.

6.1 Consultas históricas

Función	Devuelve
`fn_historial_usuario(p_usuario, p_desde, p_hasta, p_tabla)`	Todos los cambios hechos por un usuario, opcionalmente en un rango y una tabla. Si el usuario no tiene cambios devuelve vacío con un aviso.
`fn_historial_columna(p_tabla, p_columna, p_desde, p_hasta, p_evento)`	Cómo ha cambiado una columna específica. El evento es opcional y no distingue mayúsculas. Si la columna nunca se configuró da un error claro.
`fn_historial_registro(p_tabla, p_pk)`	Historia completa de una fila, buscada por su PK en JSON. Funciona aunque la fila ya se haya borrado y también si su PK cambió.
`fn_linea_tiempo(p_desde, p_hasta, p_tabla, p_usuario, p_evento, p_limite)`	Línea de tiempo general con cualquier combinación de filtros. Con `p_limite` devuelve los N cambios más recientes, ordenados del más antiguo al más nuevo.

Todas devuelven las mismas columnas que `v_bitacora_detalle`: id_bitacora, fecha_hora, nombre_tabla, nombre_columna, tipo_evento, id_registro, valor_anterior, valor_nuevo, nombre_usuario, nombre_estacion, direccion_ip, id_configuracion, configuracion_activa, id_usuario e id_estacion.

Ejemplos

```
SELECT * FROM fn_historial_usuario('demo_supervisor');
SELECT * FROM fn_historial_usuario('demo_supervisor', '2026-11-01', '2026-12-01');
SELECT * FROM fn_historial_columna('cliente', 'limite_credito');
SELECT * FROM fn_historial_columna('cliente', 'limite_credito', p_evento => 'update');
SELECT * FROM fn_historial_registro('cliente', '{"id_cliente": 3}');
SELECT * FROM fn_linea_tiempo(p_limite => 100);
SELECT * FROM fn_linea_tiempo(p_tabla => 'cliente', p_usuario => 'demo_cajero');
```

Resultado del ejemplo (script 14) para la historia del cliente 3, que fue creado por el cajero y luego eliminado por el supervisor:

Aunque la fila ya no existe en cliente, sus valores siguen en la bitácora. El cambio de teléfono que hizo el cajero no aparece porque esa columna no se seleccionó.

6.2 Reportes

Función	Contenido
fn_reporte_por_usuario(desde, hasta)	Totales por usuario, operaciones y tablas afectadas.
fn_reporte_por_columna(desde, hasta, tabla)	Totales por tabla y columna, usuarios y último cambio.
fn_reporte_actividad_diaria(desde, hasta, tabla)	Recuento diario por tipo de operación.
fn_reporte_estado_auditoria()	Configuración, presencia del trigger y registros en bitácora.
fn_reporte_estado_triggers()	Presencia, habilitación en sesión normal y modo del trigger.

Resumen por usuario de la ejecución histórica del ejemplo.

Usuario	Total	INSERT	UPDATE	DELETE	Tablas
demo_cajero	9	9	0	0	1
demo_supervisor	6	0	3	3	1

6.3 Exportación a CSV y JSON

Ejecuta sql/12_exportar_reportes.sql con psql en Ubuntu. Los archivos se guardan en el equipo cliente. Usa un rol de consulta o de administración y ejecuta el archivo sin una transacción externa.

```
padre=$(mktemp -d reportes-XXXXXX)
psql -X -d autoaudit -v dir="$padre/noviembre" \
  -v desde=2026-11-01 -v hasta=2026-12-01 \
  -v usuario=demo_supervisor -v tabla=cliente \
  -v columna=limite_credito -f sql/12_exportar_reportes.sql
```

El exportador crea una carpeta privada nueva. La carpeta padre debe existir, estar bajo tu control y tener una ruta sin enlaces simbólicos. Si omites dir, el destino es reportes. El exportador rechaza destinos existentes, rutas con segmentos .. y prefijos de programa.

Archivo	Contenido
linea_tiempo.csv / .json	Cambios del período con el detalle completo.
resumen_usuarios.csv	Resumen por usuario.
resumen_columnas.csv	Resumen por columna.
actividad_diaria.csv	Actividad por día.
estado_auditoria.csv	Estado de configuración. Conserva su encabezado original.
historial_usuario.csv	Se genera cuando se indica usuario.
historial_columna.csv	Se genera cuando se indican tabla y columna.

Las fechas delimitan un intervalo con inicio inclusivo y fin exclusivo. El reporte de estado refleja la configuración de la misma instantánea que las demás consultas. Todo el lote se lee con REPEATABLE READ READ ONLY y se escribe en UTF-8.

Los CSV incluyen encabezados. El JSON es un arreglo con las mismas columnas de la línea de tiempo y devuelve un arreglo vacío cuando no hay cambios en el período. Las muestras previas se conservan en evidencias/reportes_ejemplo/.

Si psql devuelve un error, considera incompleto el lote y reintenta con un directorio nuevo. Importa los campos de CSV como texto al abrirlos en una hoja de cálculo. El JSON reúne el período completo en memoria, por lo que conviene acotar las fechas en bitácoras grandes.

Historial del cliente 3 en la demostración

Extracto histórico de los valores conservados tras el DELETE.

Columna	Operación	Valor anterior	Valor nuevo	Usuario
categoria	INSERT		A	demo_cajero
limite_credito	INSERT		10000.00	demo_cajero
nombre	INSERT		Distribuidora Sur	demo_cajero
categoria	DELETE	A		demo_supervisor
limite_credito	DELETE	10000.00		demo_supervisor
nombre	DELETE	Distribuidora Sur		demo_supervisor

7. Seguridad y control de acceso

- **Configuración restringida.** Solo `autoaudit_admin` o un superusuario pueden configurar y generar triggers. Las tablas de configuración no aceptan escritura directa.
- **Bitácora protegida.** Nadie, ni siquiera el administrador, puede hacer INSERT, UPDATE, DELETE o TRUNCATE directo en `bitacora`, `usuario_auditado` o `estacion_trabajo`. Unos triggers guardia bloquean cualquier intento que no venga de la función de registro.
- **Lectura separada.** El rol `autoaudit_consulta` puede leer el historial y los reportes, pero no puede escribir en la bitácora ni configurar. Un usuario sin ese rol no puede leer ni la vista ni las funciones de consulta (pruebas F1 a F6).
- **Identidad real.** El usuario se toma de `session_user`, no de un dato enviado por la aplicación, así que no se puede suplantar. Con SET ROLE se registra el login original.
- **Funciones con privilegios acotadas.** Las funciones SECURITY DEFINER fijan `search_path = pg_catalog, pg_temp` y califican todos los objetos con su esquema, lo que evita que alguien redirija las llamadas con objetos propios. Los identificadores dinámicos se construyen con `format('%I')`.
- **Atomicidad.** Si falla el registro en la bitácora, también se revierte la operación de negocio. No puede quedar un cambio sin historial.

> Un superusuario o el dueño de los objetos siempre puede desactivar triggers o modificar tablas. AutoAudit protege el historial frente a usuarios normales y administradores de auditoría, no frente a alguien con control total del servidor. Por eso no se debe dar membresía de postgres ni propiedad de objetos a los usuarios de negocio.

8. Pruebas de validación y rendimiento

Hay tres juegos de pruebas. Todos se ejecutan dentro de una transacción que termina en ROLLBACK, con aserciones que detienen el script si algo no cuadra.

Script	Resultado	Qué cubre
06_pruebas_configuracion.sql	8 escenarios	Diccionario, configuración, repetición sin duplicados, activación y desactivación, deshabilitar tabla, detección de columna eliminada.
09_pruebas_triggers.sql	40 OK	Generador, valores de INSERT/UPDATE/DELETE, NULL, PK compuesta, tablas sin PK, particiones, guardias, colisiones, errores y reversión.
13_pruebas_validacion.sql	49 OK	Validación de punta a punta del proyecto, consultas, reportes, permisos y rendimiento con volumen creciente.

8.1 Entorno de ejecución

Las pruebas se ejecutaron en un servidor PostgreSQL 18.6 de 64 bits sobre Linux x86_64, instalado desde cero con los scripts del repositorio. Los resultados completos, con fecha y versión del servidor, están en la carpeta `evidencias`. El servidor de prueba no era una instalación de Ubuntu 26.04 propiamente dicha, sino binarios oficiales de PostgreSQL 18.6 para Linux; los tiempos pueden variar en otro equipo y conviene repetir el script 13 en el servidor final.

8.2 Casos del script 13

Se crea una tabla `producto` con cinco columnas. Se auditan `precio` y `existencia` en todos los eventos, `nombre` solo en INSERT y DELETE, y `descripcion` no se audita. Participan dos usuarios de negocio, un administrador, un auditor y un usuario sin permisos.

Grupo	Comprobaciones
A. INSERT	Dos inserciones generan exactamente 6 registros (3 columnas configuradas). La columna no seleccionada no aparece. El valor anterior es NULL y el nuevo, la PK, el usuario, la fecha y la estación son correctos.
B. UPDATE	Solo se registran los cambios reales: un UPDATE que no cambia el valor y otro sobre una columna sin UPDATE configurado no generan filas. Precio pasa de 150.00 a 165.00 por el usuario correcto; existencia de 20 a 15.
C. DELETE	La fila se elimina de la tabla, pero la bitácora guarda los tres valores anteriores con valor nuevo NULL, y siguen ahí aunque la fila ya no exista.
D. Consultas	Por usuario sin mezclar a otros usuarios, usuario sin cambios, filtro por tabla, por columna y evento, orden cronológico, por registro, línea de tiempo con filtros, rangos vacíos, errores claros para columna no configurada, rango invertido, evento inválido y usuario vacío, historial de una tabla ya deshabilitada.
E. Reportes	Los totales del resumen por usuario, por columna y diario cuadran con la bitácora. El último usuario por columna es el correcto.

F. Seguridad	El auditor consulta pero no escribe ni configura. Un usuario sin rol no puede consultar. El administrador también consulta.
G. Rendimiento	Cantidades exactas con 1 000, 10 000 y 50 000 filas, comparación con y sin auditoría y tiempos de consulta con la bitácora creciendo.

8.3 Impacto en las operaciones

Se ejecutó cada operación sobre la misma cantidad de filas en una tabla sin auditoría y en otra igual con una columna auditada. Tiempos en milisegundos de una corrida:

Filas	Operación	Sin auditoría	Con auditoría	ms extra por fila	Registros en bitácora
1 000	INSERT	2.2	155.0	0.153	1 011
1 000	UPDATE	4.4	192.8	0.188	2 011
1 000	DELETE	0.5	132.0	0.132	3 011
10 000	INSERT	18.4	1 656.1	0.164	13 011
10 000	UPDATE	34.9	1 704.5	0.167	23 011
10 000	DELETE	4.1	1 240.4	0.124	33 011
50 000	INSERT	89.9	7 542.4	0.149	83 011
50 000	UPDATE	196.6	8 569.5	0.168	133 011
50 000	DELETE	21.3	6 897.9	0.138	183 011

El costo extra se mantiene entre 0.12 y 0.19 ms por fila y no crece con el tamaño de la bitácora, que pasó de 1 011 a 183 011 registros durante la prueba. En una operación normal de la aplicación, que toca una o pocas filas, la diferencia no se percibe. En cargas masivas sí se nota: insertar 50 000 filas pasa de menos de una décima de segundo a unos 7.5 segundos. Para cargas grandes planificadas se puede retirar el trigger temporalmente con `fn_autoaudit_retirar` y regenerarlo al terminar, sabiendo que esa carga no quedará auditada.

8.4 Tiempo de las consultas con volumen creciente

Consulta	3 011 registros	33 011 registros	183 011 registros
fn_historial_columna (producto.precio)	2.33 ms	1.18 ms	1.29 ms
fn_historial_usuario (pocos cambios)	2.67 ms	1.35 ms	1.24 ms
fn_linea_tiempo (últimos 100)	2.74 ms	2.42 ms	2.86 ms
fn_reporte_por_usuario (resumen completo)	4.59 ms	36.32 ms	184.06 ms

Las consultas que filtran por usuario, columna o fecha se mantienen alrededor de uno a tres milisegundos sin importar el tamaño de la bitácora, porque usan los índices del módulo 10. El plan de ejecución guardado en la evidencia muestra *Index Scan using ix_bitacora_usuario_fecha*. El reporte completo por usuario sí crece con el volumen, porque recorre toda la bitácora para sumar; con 183 mil registros tarda menos de dos décimas de segundo. Para bitácoras muy grandes se recomienda pedirlo por rango de fechas.

Durante las pruebas se encontró que la línea de tiempo tardaba unos 400 ms con 183 mil registros, porque PostgreSQL reutilizaba un plan genérico que no aprovechaba el índice. Se corrigió armando la consulta con los filtros ya resueltos y ejecutándola con parámetros, y bajó a menos de 3 ms.

8.5 Compatibilidad con los módulos anteriores

El script 09 del módulo de triggers se volvió a ejecutar con los módulos de consultas y reportes instalados y dio los mismos 40 OK. Los módulos nuevos no modifican columnas, llaves ni datos de las partes anteriores; solo agregan una vista, funciones de lectura, índices y un rol.

9. Limitaciones conocidas

- Se trabaja sobre el esquema `public`.
- TRUNCATE en tablas de negocio no es INSERT, UPDATE ni DELETE y no se audita. Se recomienda quitar ese permiso a los roles de negocio.
- Los cambios de estructura (DROP o RENAME de columnas o tablas) no se interceptan. Después de un cambio así hay que validar y regenerar.
- Si la aplicación usa un único login compartido, la bitácora registra ese login, no a la persona que usa la interfaz.
- La IP es NULL cuando la conexión es por socket local, y detrás de un proxy se registra la IP del proxy.
- Las fechas se guardan como timestamp sin zona, en hora de Guatemala.
- Las tablas sin llave primaria se auditan, pero `id_registro` queda NULL y no se puede consultar por registro.
- Un superusuario puede desactivar los triggers; la protección es frente a usuarios normales.
- Los tiempos de la sección 8 son de una sola corrida en un equipo de prueba. Deben repetirse en el servidor final con su carga real.

10. Estructura del repositorio

```
AutoAudit/
├── README.md                          Guía rápida
├── instalar.sh                        Instalación completa
├── README_modulo_triggers.md          Detalle del módulo de triggers
├── LEEME_PARA_TRIGGERS.md
├── sql/
│   ├── 01_tablas.sql                  Modelo de 7 tablas
│   ├── 03_ajustes_configuracion.sql   Restricciones y eventos
│   ├── 04_consultas_diccionario.sql   Diccionario de datos
│   ├── 05_modulo_configuracion.sql    Configuración por tabla/columna/evento
│   ├── 06_pruebas_configuracion.sql   Pruebas de configuración
│   ├── 07_funciones_bitacora.sql      Registro y protección del historial
│   ├── 08_generador_triggers.sql      Generador de triggers
│   ├── 09_pruebas_triggers.sql        Pruebas de triggers
│   ├── 10_consultas_historicas.sql    Consultas, índices y rol de consulta
│   ├── 11_reportes.sql                Reportes
│   ├── 12_exportar_reportes.sql       Exportación a CSV y JSON
│   ├── 13_pruebas_validacion.sql      Pruebas de validación y rendimiento
│   └── 14_ejemplo_uso.sql             Ejemplo completo
├── docs/
│   ├── Documentacion_Final_AutoAudit.pdf
│   ├── 02_diccionario_datos.txt
│   ├── README_configuracion.md
│   ├── análisis y revisión del módulo de triggers (.md)
│   ├── evidencias de pruebas de los módulos anteriores (.txt)
│   ├── img/                           Diagramas ER y de flujo
│   └── AutoAudit - proy - bd1 (3).pdf  Enunciado
└── evidencias/
    ├── resultado_pruebas_validacion.txt
    ├── resultado_09_con_modulo_consultas.txt
    ├── resultado_ejemplo_uso.txt
    └── reportes_ejemplo/              CSV y JSON exportados
```